the *train* set. We therefore carefully avoid overfitting by tracking the loss on the associated *valid* set, early stopping as we reach a minimum.

There is probably additional performance to be extracted by running more iterations given how continuous this iterative process appears to be. We leave as future work the design of an iterative data generation process that is less compute intensive. Indeed, we believe that a lot of computation is spent on subgoals that are not necessarily providing a lot of signal for the value function, and each iteration is quite compute intensive as it requires sampling proofs for the entire training set (which takes $\sim 20k$ GPU.hours on V100s in our current setup).

5.5 Sample Complexity

Ablation of our synthetic dataset augmentation demonstrates that synthetically generated proofs generalize to some extent and provide a noticeable uplift in performance on the *valid* set for larger models.

Table 10: Ablation of the *augmented* dataset for 160m and 700m parameters models.

Model	Performance	Perplexity	# Tokens
160m *(WebMath) raw dataset*	34.12%	1.029	16B
160m *(WebMath) augmented dataset*	34.79%	1.029	16B
700m *(WebMath) raw dataset*	40.28%	1.024	15B
700m *(WebMath) augmented dataset*	42.56%	1.024	15B

Our main motivation for including synthetic proofs in our training, beyond the relative uplift achieved, is the study of the effect of model size and training a value function on the sample complexity of our models, as we control exactly how many examples from the synthetic domain we use for training. Table 11 reports the performance on 100 synthetically generated statements (different from the train set) as well as the number of synthetic proofs present in the training set for each model (in parenthesis).

Table 11: Performance of our models on 100 test statements from our synthetic generators (run with the same parameters used to augment the training set (see table 4).

Model	9-digit addition	9-digit division	Ring equalities
160m *raw*	13% (0)	4% (0)	6% (0)
160m *augmented*	78% (100)	27% (100)	77% (100)
160m *policy+value* (iteration 1)	87% (100)	24% (100)	71% (100)
160m *policy+value* (iteration 2)	90% (100)	28% (100)	79% (100)
700m *raw*	12% (0)	5% (0)	7% (0)
700m *augmented*	76% (100)	32% (100)	82% (100)
700m *policy+value* (iteration 1)	90% (100)	40% (100)	78% (100)
700m *policy+value* (iteration 2)	**92% (100)**	**47% (100)**	**88% (100)**

This demonstrates the close (yet not perfectly correlated) relationship between sample complexity and performance in our formal reasoning setup, suggesting that sample complexity is an important driver of improved performance with formal mathematics.

More importantly it demonstrates that our models are capable of acquiring new non-trivial capabilities with a number of training examples that is compatible with manual formalization. We plan in the future to study similar learning dynamics for more challenging tasks for which we don't have a synthetic generator.

5.6 Results

We attempted to push the performance of our models by increasing both the number of expansions per proof search from $d = 128$ to $d = 256$, and the number of attempts per proofs from $a = 4$ to $a = 32$. We report the achieved performance as a function of the number of attempts per statements on the *valid* set in Table 12.

Finally, we performed a final evaluation with $d = 256$ and $a = 32$ of our 700m model *policy+value* (iteration 2) on the held-out *test* set:

Table 12: Performance of our 700m model *policy+value* (iteration 2) as we double the number of attempts a per proposition (with $d = 256$).

Attempts	Performance	Delta
$a = 2$	42.90%	
$a = 4$	47.29%	+4.39%
$a = 8$	51.26%	+3.97%
$a = 16$	54.05%	+2.99%
$a = 32$	56.50%	+2.45%

$$\text{Perf}^{test}_{a=32,e=32,d=256}(\theta_{700m}) = 56.22\%$$

6 Output

We describe in this section two projects we executed, aimed at sharing with the Metamath community results and tools based on our work.

6.1 Proof Shortening

We contributed 23 shortened proofs[11][12] of theorems to the Metamath library. These proofs were generated by the *GPT-f* automated prover. To discover shorter proofs, we sampled proofs for statements from the *set.mm* library, comparing the length of the solutions found by our models to their ground truth versions, also verifying that the shorter proofs didn't rely on additional axioms.

The reception[13] from the Metamath community was positive, proof length being a metric the community care about:

> "I had a look at the proofs—very impressive results! Especially because we had a global minimization recently, and your method found much shorter proofs nevertheless."
>
> "Any ML-based system is impressive if it can find many shorter proofs than the ones we already have. Nice work."
>
> "The shorter proof is easier to translate. It's more symmetric in that it treats A and B identically. It's philosophically more concise in that it doesn't rely on the existence of a universal class of all sets."

To our knowledge, these shortened proofs are the first effective contribution of a deep learning system to a formal mathematics library[14]

6.2 *GPT-f* Proof Assistant

We created an online proof assistant[15] to allow interactive proof constructions with the assistance of our models.

We used it to formalize more than 200 theorems and exercises. We found our models to be particularly useful to automatically generate a variety of technical low level proofsteps required in most Metamath proofs, search the library by adapting existing theorems to the format needed by the user (e.g.,

[11] `https://github.com/metamath/set.mm/pull/1547`

[12] `https://github.com/metamath/set.mm/pull/1561`

[13] `https://groups.google.com/g/metamath/c/-FNsw2wyllI`

[14] To determine whether other deep learning-based provers have made contributions to their respective libraries, we looked for such contributions in the following systems: Holist family in HOL Light, CoqGym+ASTatic in Coq, TacticToe in HOL4. In addition, we interviewed 6 experts in formal mathematics and/or deep learning applied to formal mathematics.

[15] `https://groups.google.com/g/metamath/c/DO9W2QVR-_I/m/g_rsqGj0AAAJ`

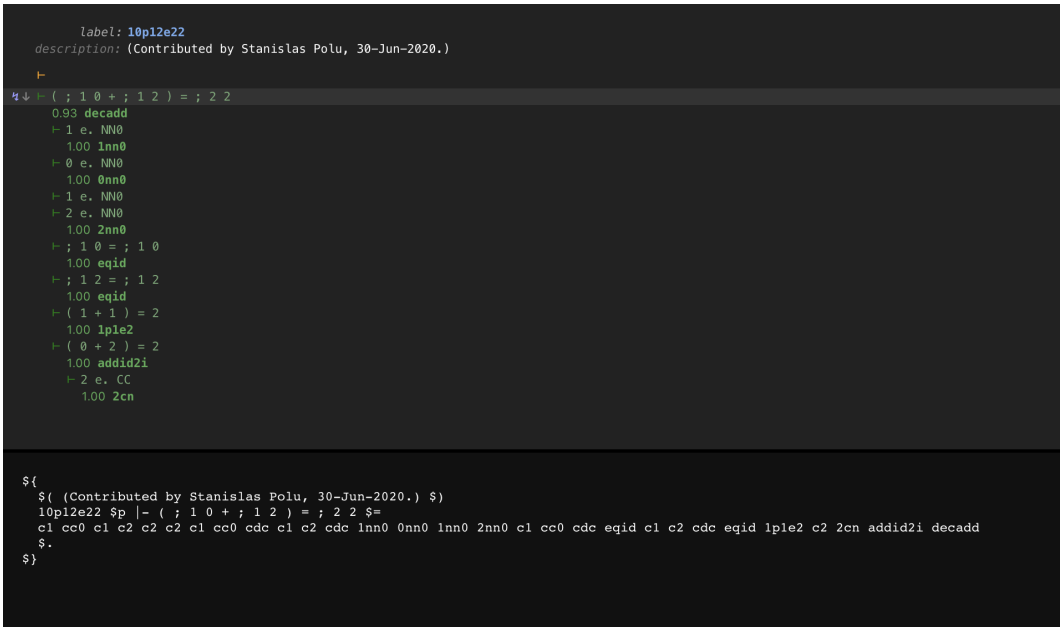

Figure 2: Screenshot of the *GPT-f* Proof Assistant

deduction form[16]) and suggest theorems to use. Even when mistaken, our models generally go for the right theorems, whose erroneous substitutions are often easy to fix by humans.

We shared the proof assistant with the Metamath community with the objective for it to be mutually beneficial, helping the community to be more productive and reciprocally helping us improve our models' accuracy by automatically gathering human feedback. We also plan to extend *GPT-f* to other formal systems.

7 Conclusion

In this paper, we present the *GPT-f* automated prover and proof assistant and show that the Transformer is suitable to formal reasoning, achieving a new state of the art result on the Metamath library. In particular we demonstrate the importance of pre-training as well as iterative training of a value function. Our results suggest that tightly coupling a deep learning system with a formal system opens up interesting opportunities for further research, with the goal of better leveraging the generative power of the former and the verification capabilities of the latter.

Acknowledgments

Szymon Sidor, Jakub Pachocki, Harri Edwards, Yura Burda and Vedant Misra inspired many of the ideas presented in this work, offering their guidance throughout the process of building *GPT-f*. Auguste Poiroux implemented the synthetic dataset generators presented in this paper, and formalized a large number of theorems using the proof assistant, providing invaluable feedback in the process. Szymon Sidor, Pranav Shyam, John Schulman, Jared Kaplan, Ryan Lowe and Jack Clark slogged through drafts of this paper, identifying errors and sources of confusion as well as providing helpful suggestions. Finally, the authors would like to thank the whole Metamath community for their support, feedback, and encouragement, in particular, David A. Wheeler for his motivating enthusiasm and Mario Carneiro for his precious help on a wide variety of technical questions.

[16] *Deduction Form and Natural Deduction* `http://us.metamath.org/mpeuni/mmnatded.html`